

Graham League Prediction Model (GLPM)

Co-founder guide — what we built, how it works, and how to run it

Last updated: July 2026

Table of contents

- 1. Executive summary
 - 2. What we implemented
 - 3. What changed from the old model
 - 4. How predictions work moving forward
 - 5. System architecture
 - 6. Data: what we need and why
 - 7. Why this is machine learning
 - 8. Training the model
 - 9. The seven rating engines
 - 10. Rating vectors and Bayesian updates
 - 11. Expected goals engine
 - 12. Match prediction markets
 - 13. Operational runbook
 - 14. Glossary
-

1. Executive summary

The **Graham League Prediction Model (GLPM)** is our new probabilistic framework for **club football** (domestic leagues). It replaces a single “team strength” number with a **seven-dimensional rating system** that estimates how many quality chances each team will create against a specific opponent — then derives every betting market from those expected goals.

Core idea: Goals are noisy. Expected goals (xG) are not. GLPM predicts xG first, then converts xG into scorelines, 1X2, BTTS, and Over/Under probabilities using the same underlying math every time.

What makes it different:

Old approach	GLPM
One composite strength index	Seven specialist ratings (attack, defence, GK, build-up, possession, pressing, finishing)
Rank teams	Estimate chance creation vs a specific opponent
Markets may use different logic	All markets from one xG pair via Dixon–Coles
Mixed data sources for clubs	Dedicated <code>glpm_*</code> schema; SportMonks primary, Wyscout enrichment

The World Cup / national-team predictor is **unchanged**. GLPM applies to **club** mode only.

2. What we implemented

2.1 End-to-end pipeline

We built a full production stack from raw match data to UI:

```

Football data (SportMonks + Wyscout)
    ↓
Layer 1 – Match ingest & storage
    ↓
Layer 2 – Feature engineering
    ↓
7 rating engines (ML training)
    ↓
Rating Vector assembly  R = [A, D, GK, BU, PO, PR, FR]
    ↓
Optional Bayesian temporal smoothing
    ↓
Expected Goals Engine → xG_home, xG_away
    ↓
Dixon–Coles score matrix → 1X2, BTTS, O/U, correct scores
    ↓
API + League Hub UI

```

2.2 Software components

Layer	What it does	Key location
Data ingest	Pull fixtures, team stats, player stats, shots from SportMonks; enrich with Wyscout PPDA and shot detail	<code>src/lib/glpm/ingestMatch.ts</code> , <code>scripts/glpm-sportmonks-*.ts</code>
Feature engineering	Turn raw stats into model-ready per-match features	<code>features/*.py</code> , <code>src/lib/glpm/layer2/</code>
Rating engines (x7)	Train hierarchical ML models per football domain	<code>models/ratings/{attack,defence,...}/</code>
Vector assembly	Combine seven primary ratings into one vector per team	<code>core/vector_assembly.py</code>
Bayesian updater	Smooth ratings over time with uncertainty	<code>core/bayesian.py</code>
xG engine	Map rating vectors + context → expected goals	<code>engine/xg_engine.py</code>
Prediction engine	Map xG → probability distributions	<code>engine/predictions.py</code>
TypeScript mirror	Same math in the Next.js app for live API predictions	<code>src/lib/glpm/engine/</code>
Database	Dedicated Supabase schema (<code>glpm_*</code> tables)	<code>supabase/migrations/030-039</code>
UI	Club predictor + league hub	<code>/predict</code> (club mode), <code>/league</code>

2.3 User-facing surfaces

- `/predict` (**Club mode**) — Pick season, home team, away team → GLPM returns xG, 1X2, BTTS, O/U, and interaction breakdown.
- `/league` (**League Hub**) — Season picker, rating leaderboards, recent results with underlying stats, upcoming fixtures with model lines.
- **API routes** — `/api/glpm/predict` , `/api/glpm/hub` , `/api/glpm/seasons` , `/api/glpm/teams` .

2.4 Specification documentation

The full statistical methodology lives in the **GOD_FILE** chapters (`docs/GOD_FILE_CHAPTER_*.md`). This guide focuses on what was built, how to operate it, and the equations that matter day-to-day.

3. What changed from the old model

3.1 Philosophy shift

The World Cup model asks: *“How strong is this team overall?”* — combining Elo, FIFA rank, momentum, form, and talent into one index.

GLPM asks: *“How many quality chances will Team A create against Team B’s specific defence, goalkeeper, and pressing style?”*

That decomposition is the biggest conceptual change. A team can be elite in attack and average in defence — a single number hides that.

3.2 Technical changes

Area	Before (club)	Now (GLPM)
Team representation	Composite / Elo-style strength	7 primary ratings → Rating Vector
Prediction input	General match form + mixed sources	Latest <code>glpm_team_rating_vectors</code> per team
xG calculation	Ad hoc or WC-derived	Interaction matrix + $\mu \cdot \exp(c \cdot \Delta S)$
Markets	Could vary by mode	Unified Dixon–Coles from same xG pair
Data schema	FBref / general tables	Dedicated <code>glpm_*</code> tables
Primary data provider	Mixed	SportMonks (primary), Wyscout (enrichment)
Explainability	Strength breakdown	Per-matchup deltas: Attack–Defence, Finishing–GK, Build–Up–Pressing, Possession–Pressing

3.3 What stayed the same

- **National / World Cup mode** on `/predict` still uses the existing World Cup strength-index flow, fixture picker, and squad XI editor.
 - The **Graham xG formula** ($\mu \cdot \exp(c \cdot \Delta S)$) is reused from the validated World Cup engine — GLPM replaces the *strength front-end* (single ΔS from Elo) with the *multi-dimensional interaction matrix*.
 - **Context adjustments** (home advantage, rest, travel, altitude) mirror proven constants from the club stadium-impact and WC rest-delta helpers.
-

4. How predictions work moving forward

Every club prediction follows the same sequence:

1. **Load the latest Rating Vector** for home and away teams (as of a given date in the season).
2. **Compute tactical matchups** — four interaction pairs per attacking side.
3. **Combine into a strength index ΔS** (weighted, capped).
4. **Apply the Graham baseline** — $xG_{base} = \mu \cdot \exp(c \cdot \Delta S)$.
5. **Apply context multipliers** — home advantage, rest days, travel, altitude.
6. **Build a Dixon–Coles score matrix** from (xG_H, xG_A) .
7. **Aggregate markets** — 1X2, BTTS, Over/Under lines, correct scores.

After each completed match:

1. Ingest new match data (SportMonks + optional Wyscout enrich).
2. Recompute Layer 2 features.
3. Retrain or refresh rating engines (scheduled, not per-match in v1).
4. Reassemble rating vectors.
5. Optionally run Bayesian update for temporal smoothing.
6. Regenerate predictions for upcoming fixtures.

Important: Predictions are only as good as the data and the freshness of rating vectors. A team with missing GK data will get a neutral (average) z-score for that dimension, which reduces edge detection but does not crash the pipeline.

5. System architecture

5.1 Four database layers

Layer 1 – Raw observations	glpm_matches, glpm_match_team_stats, glpm_match_player_s
Layer 2 – Engineered features	glpm_match_team_features, glpm_team_style_snapshots
Layer 3 – Ratings	glpm_team_primary_ratings, glpm_team_rating_vectors, glp
Layer 4 – Predictions	glpm_prediction_history, glpm_validation_logs

5.2 Modular design

Each rating engine (Attack, Defence, etc.) is **independent**. You can improve the Finishing engine without touching Defence. The xG engine consumes only the assembled Rating Vector — it does not care how individual ratings were estimated.

This mirrors the design philosophy in Chapter 1: modularity, interpretability, and incremental improvement.

5.3 Architecture diagram

Architecture diagram (see markdown source for interactive version):

```

flowchart TB
    subgraph sources [Data Sources]
        SM[SportMonks]
        WS[Wyscout]
    end

    subgraph ingest [Layer 1 & 2]
        ING[Ingest & Validate]
        FE[Feature Engineering]
    end

    subgraph ratings [Layer 3 – Rating Engines]
        A[Attack]
        D[Defence]
        GK[Goalkeeper]
        BU[Build-Up]
        PO[Possession]
        PR[Pressing]
        FR[Finishing]
        ASM[Vector Assembly]
    end

    subgraph predict [Layer 4 – Prediction]
        XG[xG Engine]
        DC[Dixon-Coles]
        API[API / UI]
    end

    SM --> ING
    WS --> ING
    ING --> FE
    FE --> A & D & GK & BU & PO & PR & FR
    A & D & GK & BU & PO & PR & FR --> ASM
    ASM --> XG
    XG --> DC
    DC --> API

```

6. Data: what we need and why

6.1 Data providers

Provider	Role	What it supplies
SportMonks	Primary	Fixtures, results, team match stats (xG, shots)
Wyscout	Secondary enrichment	PPDA refinement, shot coordinates/events, advanced analytics

Ingest is orchestrated by `src/lib/glpm/ingestMatch.ts`. Backfill scripts exist for full-season

6.2 Minimum data for rating training

To train all seven engines on a season, we need **completed matches** with:

Data category	Used by	Examples
Match results	All engines	Goals, home/away, date
Team attacking stats	Attack, Finishing	Shots, xG, box entries, big chances
Team defensive stats	Defence, Pressing	xGA, tackles, interceptions, PPDA
Possession / passing	Build-Up, Possession	Pass completion, progressive passes
Player GK stats	Goalkeeper	Saves, PSxG, goals conceded (minutes)
Shot-level detail	Finishing (ideal)	Body part, goal-mouth zone, xG per shot

6.3 What each data type does in the model

xG (expected goals) – The central process metric. Attack is trained to predict future xG creation

Shots and shot quality – Volume (how many chances) vs quality (xG per shot, big-chance rate). See

Possession and passing – Build-Up and Possession engines. Measure how teams progress the ball and

PPDA (Passes Allowed Per Defensive Action) – Pressing intensity proxy. Lower PPDA = more intense

PSxG (Post-Shot Expected Goals) – Goalkeeper shot-stopping quality. Core input for GK engine: `Goal`

Context variables – Home/away, rest days, travel distance, venue altitude. Applied at **prediction**

6.4 Data validation

Before any match enters the pipeline, validation rules check (`src/lib/glpm/validation/rules.ts`)

- Goals and xG are non-negative
- Possession percentages sum to ~100%
- Required identifiers (SportMonks IDs) are present
- Schema consistency

Failed validations are logged to `glpm_validation_logs` without blocking the entire batch.

6.5 Key database tables

Table	Purpose
<code>glpm_competitions</code> , <code>glpm_seasons</code> , <code>glpm_teams</code> , <code>glpm_players</code>	Dimensions
<code>glpm_matches</code>	Fixture spine (SportMonks)
<code>glpm_match_team_stats</code>	Full team stats per match
<code>glpm_match_player_stats</code>	Per-player stats included
<code>glpm_match_shots</code> , <code>glpm_match_events</code>	Shot and event detail
<code>glpm_match_team_features</code>	Layer 2 engineered features
<code>glpm_team_primary_ratings</code>	Long-form primary ratings
<code>glpm_team_rating_vectors</code>	Wide table: <code>r_attack</code> , <code>r_defence</code>
<code>glpm_prediction_history</code>	Archived predictions with results
<code>glpm_provider_payloads</code>	Raw JSON audit trail

7. Why this is machine learning

GLPM is not a hand-tuned formula with fixed coefficients for team quality. The **rating estimation**

7.1 The learning problem

For each rating engine, we ask:

Given this team's recent adjusted performance features, can we predict how they will perform o

That is a **supervised regression** problem:

- **Inputs (X):** Opponent-adjusted, context-adjusted feature vectors from historical matches
- **Target (y):** Forward-looking 5-match rolling mean of a process metric (e.g. non-penalty xG per
- **Output:** A latent score → calibrated to 0–100 GLPM scale

We are learning latent football abilities that are not directly observable – classic latent-vari

7.2 Models used

Condition	Estimator
< 500 training rows	Bayesian Ridge Regression – linear, with built-in uncertain
≥ 500 training rows	LightGBM – gradient-boosted trees for non-linear patterns

Every engine uses the same estimator selection logic. BayesianRidge gives us return_std for con

7.3 Hierarchical structure (why it's not one big model)

Each primary rating is built in three ML layers:

Raw adjusted features
 ↓ ML (6 component models)
Component Ratings (e.g. chance_volume, chance_quality, ...)
 ↓ ML (3 domain models)
Domain Ratings (e.g. creation, progression, situational)
 ↓ ML (1 primary model)
Primary Rating (e.g. Attack A)
 ↓ Calibration
0–100 GLPM Score

This hierarchy is itself a modelling choice: it forces the model to learn **interpretable sub-abil.**

7.4 Calibration (not ML, but essential)

Raw model outputs are mapped to the universal 0–100 scale via **empirical percentile calibration** (

- Fit on the distribution of latent scores in the training set
- Map percentiles to fixed score bands (Elite = 90–100, Average = 60–69, etc.)

This ensures a rating of 75 means the same thing across Attack, Defence, and Finishing.

7.5 Bayesian updating (probabilistic, not retraining)

After ML training produces engine outputs, an optional **Gaussian conjugate update** smooths ratings

7.6 What is *not* ML

The xG engine and Dixon–Coles market layer use **fixed, interpretable equations** with locked coeffi

8. Training the model

8.1 Prerequisites

```
# Python environment
bash scripts/ml/setup-venv.sh

# Supabase credentials in .env (for loading match frames)
# SportMonks API key (for ingest)
```

8.2 Recommended order of operations

```
# Step 0 – Ingest a full season
npm run glpm:sm-backfill -- --season-id <SEASON_SM_ID>
npm run glpm:wyscout-backfill -- --season-id <SEASON_SM_ID> # optional enrichment

# Step 1 – Train all seven rating engines
npm run glpm:attack-train -- --season-id <ID>
npm run glpm:defence-train -- --season-id <ID>
npm run glpm:goalkeeper-train -- --season-id <ID>
npm run glpm:build-up-train -- --season-id <ID>
npm run glpm:possession-train -- --season-id <ID>
npm run glpm:pressing-train -- --season-id <ID>
npm run glpm:finishing-train -- --season-id <ID>

# Step 2 – Assemble rating vectors
npm run glpm:assemble-vectors -- --season-id <ID>

# Step 3 – (Optional) Bayesian temporal smoothing
npm run glpm:bayesian-update -- --season-id <ID> --half-life 90

# Step 4 – Predict a fixture
npm run glpm:predict-from-vectors -- --home-team-id <H> --away-team-id <A> --season-id <ID>

# Step 5 – Backtest / validation
npm run glpm:validation -- --season-id <ID>
```

Each train script wraps a Python pipeline that loads Supabase data, fits models, calibrates scores, and predicts.

8.3 Synthetic training (development only)

Each engine supports `--synthetic` mode for testing without real data:

```
npm run glpm:attack-train:synthetic
```

8.4 When to retrain

Trigger	Action
New season starts	Full backfill + retrain all engines
Mid-season (weekly)	Reassemble vectors; optional Bayesian update
Major data backfill	Retrain affected engines
Model improvement	Retrain specific engine independently

9. The seven rating engines

All seven engines share the same pipeline structure:

Historical matches

- Feature engineering
- Opponent & context adjustment
- 6 Component models (ML)
- 3 Domain models (ML)
- 1 Primary model (ML)
- GlpmCalibrator → 0–100 score

Opponent adjustment (all team-level engines):

```
s_opp = league_mean_xg_conceded / opponent_rolling_xg_conceded
feature_adj = (feature - context_offset) / s_opp
```

When defence ratings are available, `s_opp` can use `defence_rating / 50` instead of raw xGA roll

Training target (all engines):

```
y = mean(next 5 matches of [process metric])
```

9.1 Attack (A)

Question answered: *How good is this team at creating chances?*

Level	Name
Components (6)	chance_volume, chance_quality, ball_progression, territorial_cor
Domains (3)	creation, progression, situational
Primary	Attack (A)

Key input features:

Component	Features
Chance volume	shots_p90, box_entries_p90, big_chances_p90, touches_in_box_p90
Chance quality	xg_per_shot, big_chance_pct, central_shot_pct, avg_shot_distance
Ball progression	prog_pass_rate, prog_carry_rate, final_third_entry_rate
Territorial control	field_tilt, territory_pct, final_third_occupancy
Transition threat	transition_xg_per_recovery, counter_efficiency, fast_break_rate
Set piece threat	set_piece_xg_per_match, set_piece_shot_rate

Primary training target: Future adjusted non-penalty xG per 90 (npxg_p90_adj).

9.2 Defence (D)

Question answered: *How good is this team at preventing chances?*

Level	Name
Components (7)	chance_suppression, defensive_organisation, transition_defence,
Domains (3)	prevention, protection, control
Primary	Defence (D)

Key input features:

Component	Features (higher = better defence)
Chance suppression	shots_suppression, xga_per_opp_poss, big_chances_conceded_i
Defensive organisation	def_actions_per_poss, interception_rate, tackle_share
Transition defence	transition_xga_per_loss, fast_break_prevention_rate
Box protection	blocks_per_box_entry, clearances_per_box_entry
Set piece defence	set_piece_xga_suppression, corner_xga_suppression
Pressing	ppda_inv, high_turnovers_p90, press_duel_rate
Territorial control	field_tilt_against_suppression, opp_box_entry_suppression

Primary training target: Future adjusted xGA suppression (xga_suppression_adj).

Concession rates are inverted: $\text{scale} / (1 + \text{rate})$ so higher values mean better defence.

9.3 Goalkeeper (GK)

Question answered: *How much does this team's goalkeeper prevent beyond expected?*

This is a **player-level** engine aggregated to team level by minutes played.

Level	Name
Components (5)	shot_stopping, area_command, distribution, sweeper, penalty
Domains (2)	goal_prevention, goalkeeper_involvement
Primary	Goalkeeper (GK)

Core metric:

Goals Prevented = PSxG – Goals Conceded

Penalty save rate shrinkage (small-sample stabilisation):

$$\theta^* = (n / (n + k)) \cdot \hat{\theta} + (k / (n + k)) \cdot \mu_0$$

where $k = 8$, $\mu_0 \approx 25\%$ (league prior save rate)

Primary training target: Future adjusted goals prevented (goals_prevented_adj).

9.4 Build-Up (BU)

Question answered: *How well does this team progress the ball under pressure?*

Level	Name
Components (6)	ball_progression, vertical_line_breaking, press_resistance, secu
Domains (3)	progression, retention, distribution
Primary	Build-Up (BU)

Primary training target: Future adjusted final-third entry rate (final_third_entry_rate_adj).

Relevant in xG engine as **Build-Up vs Opponent Pressing** matchup.

9.5 Possession (P0)

Question answered: *How well does this team control the game with the ball?*

Level	Name
Components (6)	possession_security, ball_circulation, territorial_dominance, sp
Domains (3)	ball_retention, territorial_control, possession_control
Primary	Possession (P0)

Primary training target: Future adjusted possession percentage (possession_pct_adj).

Relevant in xG engine as **Possession vs Opponent Pressing** matchup.

9.6 Pressing (PR)

Question answered: *How effectively does this team win the ball back high up the pitch?*

Level	Name
Components (6)	high_press, mid_block_press, counter_press, recovery_efficiency,
Domains (3)	press_intensity, ball_recovery, press_effectiveness
Primary	Pressing (PR)

PPDA style thresholds (aligned with tactical style snapshots):

Style	PPDA
High press	≤ 9
Mid-block	9 – 14
Low block	> 14

Primary training target: Future adjusted inverse PPDA (ppda_inv_adj).

Pressing appears in **two** xG interaction pairs (vs Build-Up and vs Possession).

9.7 Finishing (FR)

Question answered: *Does this team convert chances better or worse than expected?*

Level	Name
Components (6)	shot_accuracy, shot_technique, finishing_efficiency, clinical_fi
Domains (3)	shot_execution, chance_conversion, finishing_composure
Primary	Finishing (FR)

Core metric:

Finishing over/under-performance = Goals - xG

We deliberately do **not** train on raw goals alone – that confounds chance quality with conversion

Primary training target: Future adjusted goals-minus-xG (goals_minus_xg_adj).

Uses Wyscout shot metadata (body part, goal-mouth zones) when available for richer component fea

9.8 Rating scale (all engines)

After calibration, all primary ratings use the same interpretable scale:

Score	Band	Meaning
90 – 100	Elite	Top ~5% of league
80 – 89	Excellent	Top ~15%
70 – 79	Strong	Above average
60 – 69	Average	League median band
50 – 59	Below average	
40 – 49	Poor	
< 40	Very poor	Bottom ~5%

The “Average” band starts at **60** – this is the centering point used in the xG engine (rating_cen

10. Rating vectors and Bayesian updates

10.1 The Rating Vector

Once all seven primaries are estimated, they are assembled into a single vector per team per date:

$$R = [A, D, GK, BU, PO, PR, FR]^T$$

Stored in `glpm_team_rating_vectors` as columns `r_attack`, `r_defence`, `r_goalkeeper`, `r_build_`

Each dimension carries metadata: confidence, variance, matches used, recent trend (up/down/flat)

The League Hub “Overall” rating shown in leaderboards is the **mean of all seven primaries** from the

10.2 Z-score conversion (for xG engine)

Before entering the interaction matrix, each rating is converted to a centered z-score:

$$z = (r - 60) / 20$$

Missing or NaN ratings default to $z = 0$ (league average, neutral).

10.3 Bayesian temporal update

After vector assembly, an optional pass smooths ratings over time using Gaussian conjugate updates:

Prior variance inflation (time decay – stale evidence fades):

$$\sigma^2_{\text{prior}} \leftarrow \sigma^2 \cdot \exp(\lambda \cdot \Delta t)$$

where $\lambda = \ln(2) / 90$ days (variance doubles every 90 days, capped at 25×)

Observation update (new engine output arrives):

$$\tau_{\text{post}} = \tau_{\text{prior}} + \text{confidence_obs} / \sigma^2_{\text{obs}}$$

$$\mu_{\text{post}} = (\tau_{\text{prior}} \cdot \mu_{\text{prior}} + \tau_{\text{obs}} \cdot \mu_{\text{obs}}) / \tau_{\text{post}}$$

Initial prior at season start: $\mu = 50$, $\sigma^2 = 400$ ($\sigma = 20$) per dimension.

Confidence formula:

$$\text{confidence} = (1 / (1 + \sigma)) \cdot \text{clip}(\text{matches_used} / 20, 0.2, 1.0)$$

This ensures a team with 3 matches gets a lower-confidence rating than one with 20, and uncertain

11. Expected goals engine

The xG engine is the bridge between team quality (Rating Vector) and match probabilities. It reu

11.1 Interaction matrix

For each attacking side, four tactical matchups are evaluated:

Pair	Delta formula	Weight
Attack vs Defence	$\Delta_{\text{AD}} = z_{\text{attack}} - z_{\text{opp_defence}}$	0.40
Finishing vs Goalkeeper	$\Delta_{\text{FG}} = z_{\text{finishing}} - z_{\text{opp_gk}}$	0.25
Build-Up vs Pressing	$\Delta_{\text{BP}} = z_{\text{build_up}} - z_{\text{opp_pressing}}$	0.20
Possession vs Pressing	$\Delta_{\text{PP}} = z_{\text{possession}} - z_{\text{opp_pressing}}$	0.15

Home attacking uses home team's z-scores vs away team's defensive z-scores. **Away attacking** rever

Strength index:

$$\Delta S_{\text{raw}} = 0.40 \cdot \Delta_{\text{AD}} + 0.25 \cdot \Delta_{\text{FG}} + 0.20 \cdot \Delta_{\text{BP}} + 0.15 \cdot \Delta_{\text{PP}}$$

$$\Delta S = \text{clip}(\Delta S_{\text{raw}}, -3, +3)$$

Weights sum to 1.0. The cap at ± 3 prevents extreme ratings from producing unrealistic xG.

11.2 Baseline xG (Graham engine)

$$\text{xG_base} = \mu \cdot \exp(c \cdot \Delta S)$$

Parameter	Default	Meaning
μ	1.35	Baseline xG per team in an average matchup
c	0.28	Strength exponent – a full Attack–Defence edge ($\Delta S \approx 0.40$) -

Example: If the home team has $\Delta S = +1.0$ (clear overall edge):

$$\text{xG_base} = 1.35 \cdot \exp(0.28 \times 1.0) = 1.35 \cdot 1.323 = 1.79$$

11.3 Context multipliers

Applied per side after baseline xG:

$$\text{xG_final} = \text{clip}(\text{xG_base} \times \text{home_mult} \times \text{away_mult}, \quad 0.15, \quad 4.5)$$

Factor	Home	Away	Notes
Home advantage	$\times 1.12$	$\times 1.00$	Disabled at n
Rest days	piecewise	same	≥ 3 days $\rightarrow 1.0$
Congestion	$\times 0.97$ extra	same	When rest ≤ 4
Travel	–	$\times 0.95$ if >1500 km; $\times 0.98$ if >500 km	Away team onl
Altitude	$\times 1.00$	$\times 0.97$ if venue >1000 m	Away team onl

Competition-specific baselines can override μ via `context.competition_mu`.

11.4 Complete xG worked example

Suppose:

- Home Attack $z = +1.5$, Away Defence $z = -0.5 \rightarrow \Delta_{AD} = +2.0$

- Home Finishing $z = +0.5$, Away GK $z = +0.0 \rightarrow \Delta_{FG} = +0.5$
- Home Build-Up $z = +0.0$, Away Pressing $z = +1.0 \rightarrow \Delta_{BP} = -1.0$
- Home Possession $z = +0.5$, Away Pressing $z = +1.0 \rightarrow \Delta_{PP} = -0.5$

$$\begin{aligned}\Delta S_{\text{raw}} &= 0.40(2.0) + 0.25(0.5) + 0.20(-1.0) + 0.15(-0.5) \\ &= 0.80 + 0.125 - 0.20 - 0.075 \\ &= 0.65\end{aligned}$$

$$xG_{\text{base}} = 1.35 \cdot \exp(0.28 \times 0.65) = 1.35 \cdot 1.197 = 1.62$$

$$xG_{\text{home}} = 1.62 \times 1.12 \text{ (home advantage)} = 1.81$$

The same process runs for the away team attacking the home defence.

12. Match prediction markets

Given (xG_H, xG_A) , all markets are derived from a single **Dixon-Coles Goal Probability Matrix**.

12.1 Score matrix

Independent Poisson base with low-score correlation correction:

$$P(h, a) \propto \text{Poisson}(h; \lambda_H) \cdot \text{Poisson}(a; \lambda_A) \cdot \tau(h, a)$$

where $\lambda_H = xG_H$, $\lambda_A = xG_A$, and grid covers goals $h, a = 0 \dots 9$ (10×10 matrix, renormalised).

Dixon-Coles τ correction ($\rho = -0.13$ default):

Score (h, a)	τ factor
(0, 0)	$1 - \lambda_H \cdot \lambda_A \cdot \rho$
(0, 1)	$1 + \lambda_H \cdot \rho$
(1, 0)	$1 + \lambda_A \cdot \rho$
(1, 1)	$1 - \rho$
All others	1

$\rho < 0$ reflects the empirical fact that 0-0 and 1-1 occur more often than independent Poisson pre

12.2 Derived markets

Market	Formula
Home win	$\sum P(h,a)$ where $h > a$
Draw	$\sum P(h,a)$ where $h = a$
Away win	$\sum P(h,a)$ where $h < a$
BTTS Yes	$\sum P(h,a)$ where $h > 0$ AND $a > 0$
Over 2.5	$\sum P(h,a)$ where $h + a > 2.5$
Correct score	Individual cell $P(h,a)$

Over/Under lines computed: 0.5, 1.5, 2.5, 3.5, 4.5.

All predictions are archived in `glpm_prediction_history` with the full score matrix for audit :

Model versions: `glpm_xg_v1` (xG engine), `glpm_pred_v1` (Dixon-Coles layer).

13. Operational runbook

13.1 Day-to-day (during a season)

Task	Command / action
Ingest last round's matches	<code>npm run glpm:sm-ingest -- --match-id <ID></code>
Wyscout enrich (if needed)	<code>npm run glpm:wy-enrich -- --match-id <ID></code>
Refresh rating vectors	<code>npm run glpm:assemble-vectors -- --season-id <ID></code>
Bayesian smooth	<code>npm run glpm:bayesian-update -- --season-id <ID></code>
View hub	Open /league in the app

13.2 Start of season

- .. Create season in Supabase (glpm_seasons)
- !. Backfill all prior-season matches for training data
- !. Train all seven engines
- !. Assemble opening-day vectors
- !. Generate predictions for Round 1 fixtures

13.3 Environment variables required

Variable	Purpose
SUPABASE_URL / SUPABASE_SERVICE_ROLE_KEY	Database access for ingest and training
SPORTMONKS_API_KEY	Primary match data
WYSCOUT_* credentials	Optional enrichment

13.4 Key file reference

Area	Path
Specification (full methodology)	docs/GOD_FILE_CHAPTER_*.md
Python xG engine	engine/xg_engine.py, engine/interactions.py, eng
TypeScript xG engine (API)	src/lib/glpm/engine/
Rating pipelines	models/ratings/{domain}/pipeline.py
Vector assembly	core/vector_assembly.py
Bayesian update	core/bayesian.py
Ingest orchestration	src/lib/glpm/ingestMatch.ts
Club predictor UI	src/components/glpm/GlpmClubPredictor.tsx
League hub	src/app/league/page.tsx
Tests	tests/glpm/test_*.py, src/lib/glpm/engine/engine

13.5 Validation and backtesting

`npm run glpm:validation -- --season-id <ID>`

Logs prediction accuracy metrics and writes to `glpm_validation_logs`. Use this after any model

14. Glossary

Term	Definition
GLPM	Graham League Prediction Model
Rating Vector (R)	[A, D, GK, BU, PO, PR, FR] – seven primary ratings for one team
Primary Rating	Top-level 0–100 score for one football domain (e.g. Attack)
Domain Rating	Intermediate aggregation of two component ratings
Component Rating	Most granular latent ability estimated from raw features
xG	Expected Goals – quality-weighted chance count
PSxG	Post-Shot Expected Goals – xG after shot placement/execution
PPDA	Passes Allowed Per Defensive Action – pressing intensity proxy
ΔS	Strength index – weighted sum of tactical matchup z-score components
Dixon–Coles	Poisson score model with low-score correlation correction (Dixon & Coles, 1997)
GlpmlCalibrator	Maps ML latent scores → universal 0–100 percentile scale
Layer 1 / 2 / 3 / 4	Raw data / Features / Ratings / Predictions database tiers
sm_id	SportMonks entity ID – primary key for GLPM tables
Bayesian update	Gaussian conjugate smoothing of rating vectors over time
Process metric	Underlying football behaviour (xG, xGA, goals–xG) – not model output

Quick equation reference

```

# Rating training (each engine layer)
y = mean(next 5 matches of process metric)
latent = ML(adjusted_features)
score = GlpmCalibrator(latent) →  $\in [0, 100]$ 

# Opponent adjustment
feature_adj = (feature - context_offset) / s_opp

# Z-score (for xG engine)
z = (rating - 60) / 20

# Interaction matrix
 $\Delta S = 0.40 \cdot (z_A - z_{D\_opp}) + 0.25 \cdot (z_{FR} - z_{GK\_opp})$ 
 $+ 0.20 \cdot (z_{BU} - z_{PR\_opp}) + 0.15 \cdot (z_{P0} - z_{PR\_opp})$ 
 $\Delta S = \text{clip}(\Delta S, -3, +3)$ 

# Expected goals
xG_base =  $1.35 \cdot \exp(0.28 \cdot \Delta S)$ 
xG = clip(xG_base × context_multiplier, 0.15, 4.5)

# Score matrix (Dixon-Coles)
 $P(h,a) \propto \text{Poisson}(h|\lambda_H) \cdot \text{Poisson}(a|\lambda_A) \cdot \tau(h,a; \rho=-0.13)$ 

# Bayesian update
 $\sigma^2_{\text{prior}} \leftarrow \sigma^2 \cdot \exp(\ln(2)/90 \cdot \Delta t)$ 
 $\mu_{\text{post}} = (\tau_{\text{prior}} \cdot \mu_{\text{prior}} + \tau_{\text{obs}} \cdot \mu_{\text{obs}}) / (\tau_{\text{prior}} + \tau_{\text{obs}})$ 

```

For the full statistical derivation of each engine, see docs/GOD_FILE_CHAPTER_1.md through dc